



معماری کامپیوتر

بهار ۱۴۰۵

استاد: دکتر سربازی آزاد

محسن صلاح - ۴۰۳۱۰۶۲۳۸

تمرین شماره ۶

۱ مقدمه

در این تمرین، یک حافظه‌ی نهان (cache) سطح L1 با نگاشت مستقیم (Direct-Mapped) با زبان Verilog پیاده‌سازی شد. همچنین در بخش دوم تمرین، این cache به پردازنده‌ی مشابه MIPS (که در تمرین قبل طراحی شده بود) متصل شد.

۲ طراحی حافظه‌ی نهان

۱.۲ ساختار و آدرس‌دهی

cache طراحی شده دارای $2^b = 16$ بلاک مستقیم‌نگاشت است ($b = 4$) که هر بلاک شامل $2^w = 4$ کلمه‌ی ۳۲ بیتی است ($w = 2$)؛ در نتیجه ظرفیت کل کش $2^{b+w} = 64$ کلمه است. آدرس تولیدشده توسط پردازنده سه بایت دارد:

جدول ۱: تقسیم‌بندی آدرس ۳۲ بیتی

کاربرد	تعداد بیت	فیلد	بیت‌ها
شناسه‌ی بلاک در حافظه‌ی اصلی	$c = 24$	Tag	[۳۱ : ۸]
انتخاب یکی از ۱۶ بلاک کش	$b = 4$	Block	[۷ : ۴]
انتخاب یکی از ۴ کلمه در بلاک	$w = 2$	Word	[۳ : ۲]
Byte Offset (همواره صفر؛ دسترسی‌ها word-aligned هستند)	۲	—	[۱ : ۰]

این تقسیم‌بندی با Bit-Slicing ترکیبی پیاده‌سازی شده و از هیچ عملیات حسابی استفاده نمی‌کند. آدرس پایه‌ی بلاک با صفر کردن چهار بیت کم‌ارزش محاسبه می‌شود:

```

1 wire [23:0] c_tag = CPU_Addr[31:8];
2 wire [3:0] c_idx = CPU_Addr[7:4];
3 wire [1:0] c_woff = CPU_Addr[3:2];
4 wire [31:0] c_baddr = {CPU_Addr[31:4], 4'b0};
5 wire hit = valid[c_idx] && (tag_arr[c_idx] == c_tag);

```

سیگنال hit به صورت ترکیبی محاسبه می‌شود: خط مربوطه باید هم valid باشد و هم tag ذخیره‌شده‌اش با tag آدرس جاری تطابق داشته باشد. بیت valid برای هر خط مشخص می‌کند که محتوای آن معتبر است یا نه. در

ابتدا همه‌ی این بیت‌ها صفر هستند و تنها پس از اولین واکنشی از حافظه‌ی اصلی یک می‌شوند. در صورت hit، فیلد c_woff مشخص می‌کند کدام ۳۲ بیت از ۱۲۸ بیت خط به پردازنده برگردانده شود. به عنوان مثال اگر c_woff برابر صفر باشد، بیت‌های [۰ : ۳۱] خط انتخاب می‌شوند. در غیر این صورت باید از حافظه‌ی اصلی واکنشی شود.

۲.۲ ماشین حالت

واسط cache از دو بخش تشکیل شده است: از سمت پردازنده، آدرس، نوع دسترسی (خواندن یا نوشتن) و در صورت نوشتن داده‌ی جدید دریافت می‌شود؛ cache هم داده‌ی خوانده‌شده و سیگنالی برای اعلام اتمام عملیات (CPU_Done) به پردازنده برمی‌گرداند. از سمت حافظه‌ی اصلی نیز cache می‌تواند درخواست خواندن یا نوشتن یک بلاک ۱۲۸ بیتی کامل صادر کند و منتظر تأیید (MM_Done) بماند.

کنترل این بخش با یک Finite State Machine (FSM) شش‌حالت مدیریت می‌شود. در یک دسترسی خواندن، اگر آدرس در cache باشد (hit)، داده بلافاصله به صورت ترکیبی به پردازنده بازگردانده می‌شود و نیازی به تغییر حالت نیست. اگر آدرس در cache نباشد (miss)، ماشین حالت فرایند واکنشی از حافظه‌ی اصلی را آغاز می‌کند: درخواست خواندن صادر می‌کند، منتظر پاسخ می‌ماند و پس از دریافت بلاک، آن را در کش نصب می‌کند. در دسترسی نوشتن نیز پس از به‌روزرسانی کش، یک نوشتن کامل بلاک به حافظه‌ی اصلی انجام می‌شود.

رفتار دقیق هر حالت به شرح زیر است:

- **S_IDLE**: این حالت، حالت پیش‌فرض است. در صورت read hit، داده‌ی درخواستی مستقیماً از آرایه‌ی cache خوانده و به پردازنده بازگردانده می‌شود. در صورت miss، ماشین حالت برای واکنشی بلاک وارد مرحله‌ی ارسال درخواست به حافظه می‌شود. در صورت write hit نیز پس از به‌روزرسانی کلمه‌ی هدف در کش، فرایند نوشتن به حافظه‌ی اصلی آغاز می‌شود.
- **S_MM_RD_PULSE**: درخواست خواندن برای یک چرخه‌ی ساعت به حافظه‌ی اصلی ارسال می‌شود و آدرس بلاک روی خروجی تنظیم می‌شود. ارسال یک چرخه‌ای تضمین می‌کند که حافظه‌ی اصلی هر درخواست را دقیقاً یک بار پردازش کند.
- **S_MM_RD_WAIT**: در این حالت ماشین منتظر پاسخ حافظه‌ی اصلی می‌ماند. از آنجا که حافظه‌ی اصلی ممکن است با تأخیر پاسخ دهد، آدرس ارسال‌شده باید تا دریافت تأیید ثابت بماند. پس از فعال شدن MM_Done، بلاک دریافتی در آرایه‌ی کش نوشته می‌شود؛ اگر درخواست اصلی خواندن بوده به S_IDLE برمی‌گردد و داده همان چرخه به پردازنده می‌رسد. اگر درخواست نوشتن بوده، کلمه‌ی هدف در بلاک دریافتی جایگزین می‌شود و بلاک به‌روزشده برای نوشتن به حافظه آماده می‌شود.
- **S_MM_WR_PULSE**: درخواست نوشتن برای یک چرخه به حافظه‌ی اصلی ارسال می‌شود. محتوای کامل بلاک به‌روزشده روی MM_DataOut قرار می‌گیرد.
- **S_MM_WR_WAIT**: مشابه حالت خواندن، آدرس و داده‌ی ارسال‌شده باید تا تأیید حافظه ثابت بمانند، چرا که حافظه‌ی اصلی ممکن است با تأخیر به درخواست پاسخ دهد. با فعال شدن MM_Done، نوشتن تأیید شده و ماشین وارد حالت بعدی می‌شود.
- **S_DONE**: نوشتن با موفقیت انجام شده است. در این حالت $CPU_Done = 1$ نگه داشته می‌شود تا پردازنده از پایان عملیات مطلع شود؛ به محض اینکه پردازنده این سیگنال را ببیند و درخواست نوشتن را پایین بیاورد، ماشین به S_IDLE بازمی‌گردد.

تمام انتساب‌های رفتاری در بلوک‌های always با عملگر $\leq$ (Non-Blocking) نوشته شده‌اند تا از وقوع Race Condition جلوگیری شود.

۳.۲ سیاست نوشتن

سیاست نوشتن در این حافظه، Write-Through است. یعنی هر تغییری که در cache اعمال می‌شود بلافاصله در حافظه اصلی هم انجام می‌شود. در write hit، تنها ۳۲ بیت مربوط به کلمه‌ی هدف در خط ۱۲۸ بیتی کش جایگزین می‌شود و بقیه‌ی خط دست‌نخورده می‌ماند؛ سپس بلاک به‌روزشده در حافظه‌ی اصلی نوشته می‌شود. در write miss نیز ابتدا بلاک از حافظه‌ی اصلی خوانده می‌شود، کلمه‌ی هدف در آن جایگزین و سپس بلاک کامل در حافظه‌ی اصلی نوشته می‌شود. با این سیاست، محتوای حافظه‌ی اصلی همواره با آخرین وضعیت cache همخوانی دارد و نیازی به Write-Back در زمان تخلیه‌ی بلاک نیست.

۳ اتصال کش به پردازنده

۱.۳ معماری کلی

در این بخش، پردازنده و دو نمونه از ماژول کش در یک ماژول بالاسطح (Wrapper) کنار هم قرار می‌گیرند. دلیل استفاده از دو کش مجزا این است که جریان دستورات و جریان داده کاملاً از هم مستقلند و نباید یکدیگر را مسدود کنند؛ کش دستور (icache) دستور بعدی را واکنشی می‌کند در حالی که کش داده (dcache) تنها هنگامی فعال می‌شود که دستور جاری نیاز به دسترسی به حافظه دارد.

icache همواره در حال خواندن است و آدرس بایستی دستور جاری را به عنوان ورودی دریافت می‌کند. اما dcache تنها در دستورات LW و SW و آن هم فقط پس از اینکه icache دستور را تحویل داده باشد فعال می‌شود. این گیت کردن ضروری است، چرا که اگر dcache پیش از آماده شدن دستور فعال شود، آدرسی که از دستور ناقص استخراج شده مقدار درستی ندارد و رفتار اشتباه خواهیم داشت.

از آنجا که پردازنده آدرس‌ها را به صورت کلمه‌ای خروجی می‌دهد، اما cache ها به آدرس بایستی نیاز دارند، این تبدیل در Wrapper انجام می‌شود:

$$\text{Byte Address} = \{\text{WordAddr}[29:0], 2'b00\}$$

که معادل شیفت دو بیت به چپ است و عملاً همان آدرس بایستی اصلی را بازتولید می‌کند.

۲.۳ منطق توقف

پردازنده یک ورودی توقف خارجی دارد که در صورت فعال بودن، ثبات PC ثابت می‌ماند و نوشتن در بانک ثبات هم متوقف می‌شود. این سیگنال زمانی فعال است که حداقل یکی از دو کش هنوز آماده نباشد:

$$\text{total_stall} = \neg \text{icache_done} \vee (\text{dcache_active} \wedge \neg \text{data_done_flag})$$

dcache_active زمانی یک است که دستور جاری نیاز به دسترسی به حافظه‌ی داده داشته باشد (SW یا LW) و icache هم دستور را تحویل داده باشد. اگر دستور هنوز واکنشی نشده باید صبر کرد. اما برای جمله‌ی دوم یک مشکل وجود دارد. پس از اتمام یک دستور SW، کش در حالت S_DONE می‌ماند و CPU_Done = ۱ را نگه می‌دارد تا پردازنده از پایان عملیات مطلع شود. اگر دستور بعدی یک LW باشد، Wrapper این CPU_Done = ۱ را می‌بیند و آن را به حساب دسترسی LW جاری می‌گذارد، در حالی که dcache هنوز درخواست جدید را شروع نکرده است.

برای حل این مشکل از یک رجیستر data_done_flag استفاده کردیم. این رجیستر هر بار که دستور جدیدی شروع می‌شود (یعنی لحظه‌ای که توقف رفع می‌شود) ریست شده و تنها زمانی ۱ می‌شود که CPU_Done برای دسترسی جاری آمده باشد:

```

1 always @(posedge Clk or posedge Rst) begin
2     if (Rst)
3         data_done_flag <= 1'b0;
4     else if (!total_stall)
5         data_done_flag <= 1'b0;
6     else if (dcache_done && dcache_active)
7         data_done_flag <= 1'b1;
8 end

```

با این روش، CPU_Done دستورات قبلی روی دستور جاری تأثیر نمی‌گذارد. خروجی PC نیز به طور مستقیم از InstrAddr پردازنده می‌آید و آدرس کلمه‌ای دستور جاری را نشان می‌دهد.

۴ اجرای آزمون

هر دو بخش را با تست‌بنچ‌های داده شده بررسی کردیم و نتیجه‌ی آنها به این صورت می‌باشد:

```

mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge
/mnt/D/SUT/Term_4/CA/ca4042-judge
mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge$ ./validate-verilog.sh ./HW6/cache-tb.v ../HW/P-HW-6/HW6-Q1-403106238.v
ACCEPTED
500 / 500
./HW6/cache-tb.v:177: $finish called at 163445 (1s)
mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge$ ./validate-verilog.sh ./HW6/cpu-tb.v ../HW/P-HW-6/HW6-Q2-403106238.v
fib(10) = exp: 55 real: 55
fib(11) = exp: 89 real: 89
product = exp: 4895 real: 4895
base3 digits LSD first:
2 2 0 1 0 2 0 2
base3 = 20201022
ACCEPTED
208 / 208
./HW6/cpu-tb.v:371: $finish called at 27074 (1s)
mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge$

```

شکل ۱: خروجی آزمون‌های cache-tb و cpu-tb